

Software Architecture and Design Specification

Online Internship & Placement Preparation Portal

Version: 1.0

Team No: 14

Course: Software Engineering

Date: 15-09-2026

Team Members

SRN	Name	Contribution
PES2UG24CS043	Akhila T	Architecture, SAD documentation
PES2UG24CS022	Abhinav Ranjith Das	Component diagram, API design
PES2UG24CS023	Abhirama H Y	Sequence diagrams, security architecture
PES2UG24CS039	Aishwarya	Traceability, review and formatting

Revision History

Version	Date	Author	Change Summary	Approval
1.0	15-09-2026	Team 14	Initial draft of the Software Architecture and Design Specification	Pending

Approvals

Role	Name	Signature / Date
Course Coordinator		

Table of Contents

- 1. Introduction
- 2. Document Overview
- 3. Architecture
- 4. Design
- 5. Appendices

1. Introduction

1.1 Purpose

This document describes the software architecture and design of the Online Internship & Placement Preparation Portal. It follows on from the SRS and explains how the system is put together internally — its components, the pattern that ties them together, the technology choices, and the main flows through the system — so that anyone picking up the implementation has a shared picture to work from.

1.2 Scope

Covers the portal's core services: registration and authentication, profile and resume management, placement/internship drives, mock tests, interview preparation, notifications, and TPO reporting. It does not cover the institution's own ERP or HR systems, which stay outside this project.

1.3 Audience

Developers (us), the course instructor/evaluator, and anyone reviewing or maintaining the project after submission.

1.4 Definitions

TPO, OTP, JWT, API, TLS, RBAC, CGPA, STRIDE, RTM — expanded where first used and listed again in the glossary (Section 5.1).

2. Document Overview

2.1 How to Use This Document

Section 3 covers the architecture: the goals driving it, the components and how they relate, the pattern chosen and why, the technology stack, risks, and the security threat model. Section 4 goes one level down into design: sequence diagrams for two key flows, the API contracts, error handling and monitoring, and UX notes. Section 5 has the glossary, references and tools used.

2.2 Related Documents

Software Requirements Specification (SRS) for this project, the Software Test Plan (STP, to be authored separately), and the Requirements Traceability Matrix (RTM) included in both the SRS and this document.

3. Architecture

3.1 Goals & Constraints

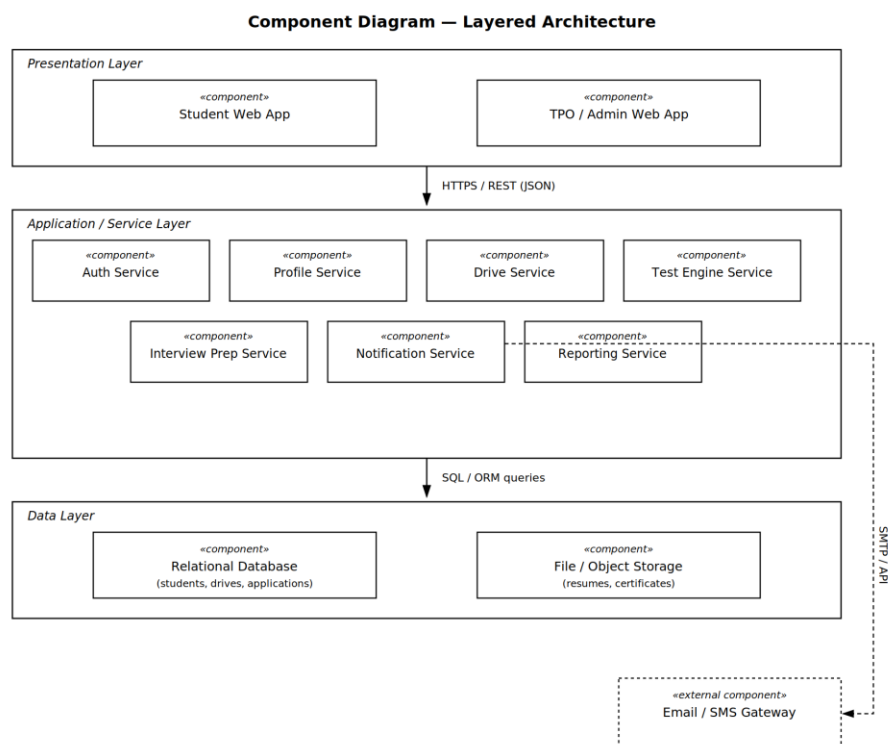
Goals: keep student data secure, keep the portal usable during high-traffic periods (deadline days), and keep the codebase simple enough for a small student team to build and maintain within a semester.

Constraints: no budget for paid infrastructure or licensing, must run on modest cloud hosting, and must be usable over the campus network which can be slow at times.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concern
Students	Data privacy, ease of use, fast page loads
TPO / Placement Cell	Accuracy of eligibility filtering, reliable reporting
Recruiters	Getting a clean shortlisted-candidate list
Course Evaluators	Clear architecture, traceability to requirements
Development Team	Maintainable, modular codebase within project timeline

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **Student Web App** — the browser-based UI a student uses to register, build a profile, browse and apply to drives, take mock tests, and track applications.
- **TPO / Admin Web App** — the browser-based UI the TPO uses to post drives, set eligibility rules, update statuses, and view reports.
- **Auth Service** — handles registration, OTP verification, login, password reset and session tokens.
- **Profile Service** — stores and serves a student's academic/personal profile and resume.
- **Drive Service** — handles posting drives, filtering by eligibility, and processing applications.
- **Test Engine Service** — runs aptitude/technical mock tests, timing and auto-grading.
- **Interview Prep Service** — serves the interview question bank and handles mock-interview slot booking.
- **Notification Service** — sends email/in-app notifications for drives, deadlines and application updates.
- **Reporting Service** — aggregates placement statistics for the TPO dashboard and CSV exports.
- **Relational Database** — stores structured data: student profiles, drives, applications, test results.
- **File / Object Storage** — stores resumes and certificates uploaded by students.
- **Email / SMS Gateway** — external service used to actually deliver OTPs and notifications.

3.5 Chosen Architecture Pattern and Rationale

We went with a layered architecture, split into logical services (auth, profile, drives, tests, interview prep, notifications, reporting) that all sit behind the same backend application rather than being deployed as separate microservices. This keeps deployment and debugging simple for a project of this size and timeline. A full microservices setup was considered and rejected — the extra operational overhead (service discovery, inter-service networking, separate deployments) isn't worth it for the number of users this portal will actually see. Keeping the services as clearly separated modules within one layer means the codebase can still be split into real microservices later if the college ever needs to scale it up.

3.6 Technology Stack & Data Stores

- **Frontend:** React (single-page app), served over HTTPS
- **Backend:** Node.js with Express, exposing REST APIs consumed by both the student and TPO front ends
- **Database:** PostgreSQL (relational) for students, drives, applications and test records
- **File storage:** cloud object storage (e.g. S3-compatible bucket) for resumes and certificates
- **Auth:** JWT-based sessions, bcrypt password hashing
- **Email/OTP delivery:** a transactional email API (e.g. SMTP relay)
- **Hosting:** a single cloud VM / small managed app platform for this project's scale, TLS terminated at the load balancer or reverse proxy

3.7 Risks & Mitigations

Risk	Mitigation
Traffic spike near an application deadline slows the portal down	Cache eligible-drive listings; keep API instances stateless so more can be added if needed
OTP/notification email is delayed or lost	Add a resend option; log delivery failures so the issue is visible
TPO edits/deletes a drive students have already applied to	Soft-delete drives and keep an audit trail instead of hard deleting
Resume storage costs or an outage in the storage provider	Use a managed, redundant object storage service rather than self-hosting file storage
A student's eligibility is computed from stale profile data	Recompute eligibility at the moment of viewing/applying, not from a cached value

3.8 Traceability to Requirements

A sample mapping from the SRS's functional requirements to the components above (full mapping is maintained alongside the RTM):

Requirement	Maps To
OIPP-F-001 / F-002 (Register & Login)	Auth Service
OIPP-F-004 / F-005 / F-006 (Profile & Resume)	Profile Service
OIPP-F-007 – F-010 (Drives & Applications)	Drive Service
OIPP-F-011 – F-013 (Mock Tests)	Test Engine Service
OIPP-F-014 – F-016 (Interview Prep)	Interview Prep Service
OIPP-F-017 / F-018 (Tracking & Notifications)	Drive Service / Notification Service
OIPP-F-019 / F-020 (TPO Dashboard & Export)	Reporting Service

3.9 Security Architecture

Threat modeling (STRIDE):

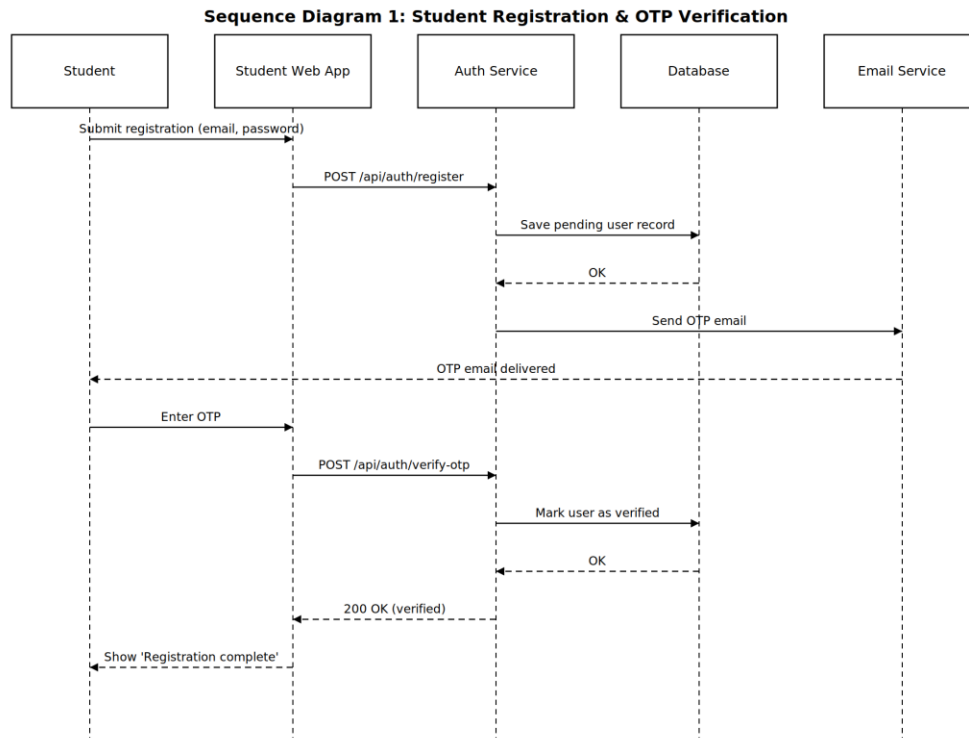
Threat	Mitigation
Spoofing	Institution-email verification + OTP at signup; signed JWT sessions
Tampering	Server-side validation of all inputs; role checked on every write, not just hidden in the UI
Repudiation	Append-only audit log for drive postings and status changes
Information Disclosure	TLS 1.2+ everywhere; bcrypt-hashed passwords; least-privilege database access
Denial of Service	Rate limiting on login/OTP endpoints; stateless API layer that can scale out
Elevation of Privilege	Role-based access control (Student / TPO / Recruiter), enforced server-side on every request

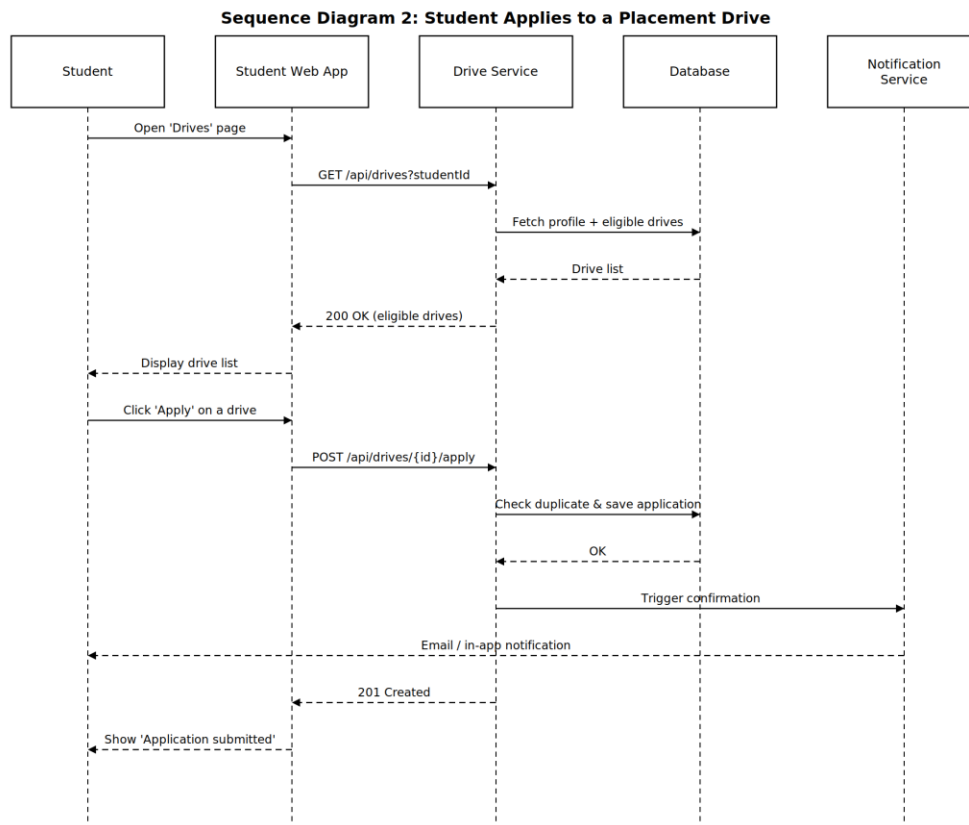
4. Design

4.1 Design Overview

The portal is built as a set of modules behind a single backend, matching the layered pattern from Section 3. Each service owns its own piece of the database schema and exposes a small REST API; the two front ends (student and TPO) are the only consumers of those APIs. The two flows below cover the paths that touch the most components and best illustrate how the layers cooperate.

4.2 UML Sequence Diagrams





4.3 API Design

Interface definitions for three of the services (at least two required):

Auth Service — Login

Endpoint	POST /api/auth/login
Request	{ email, password }
Response	{ token, studentId, expiresIn }
Errors	401 Invalid credentials · 423 Account locked (5 failed attempts)

Drive Service — Apply to a Drive

Endpoint	POST /api/drives/{driveId}/apply
Request	{ studentId } (resume/profile pulled server-side)
Response	{ applicationId, status: "applied" }
Errors	403 Not eligible · 409 Already applied · 404 Drive not found

Test Engine Service — Submit Mock Test

Endpoint	POST /api/tests/{testId}/submit
Request	{ studentId, answers: [...] }
Response	{ score, totalQuestions, submittedAt }
Errors	410 Test window closed · 409 Already submitted

4.4 Error Handling, Logging & Monitoring

All API errors return a standard JSON shape ({ error_code, message }) so the front end can handle them consistently. Logs never contain passwords, tokens, OTPs or full resume contents — only IDs and event types. Things we plan to monitor: API error rate, average response time, daily active users, drive-application failure rate, and OTP delivery failures.

4.5 UX Design

Two simple dashboards (student and TPO), each with a left-side menu. Layouts are responsive so they work on a phone as well as a laptop, since students often check drive updates on mobile. Fonts and contrast follow basic accessibility guidance, and forms auto-save drafts of longer inputs like the profile/resume form so a dropped connection doesn't lose a student's work.

4.6 Open Issues & Next Steps

- Auto-filling profile data from the institution's ERP instead of manual entry, once access is available
- Resume parsing to auto-suggest skills/keywords
- A read-only mobile app in addition to the responsive web UI
- Pulling in external internship listings through a job-board API

5. Appendices

5.1 Glossary

- **TPO** — Training & Placement Officer
- **OTP** — One-Time Password
- **JWT** — JSON Web Token, used for session authentication
- **API** — Application Programming Interface
- **TLS** — Transport Layer Security
- **RBAC** — Role-Based Access Control
- **CGPA** — Cumulative Grade Point Average
- **STRIDE** — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (a threat-modeling framework)
- **RTM** — Requirements Traceability Matrix

5.2 References

- IEEE 42010 — Systems and software engineering, architecture description
- OWASP Top 10 — common web application security risks
- NIST SP 800-160 — systems security engineering
- WCAG 2.1 — accessibility guidance referenced for the UX design

5.3 Tools

- draw.io / hand-built SVG diagrams for the component and sequence diagrams in this document
- Postman — for exercising and testing the REST APIs during development
- GitHub — version control and issue tracking for the team